

Train with Ramps, Deploy with Ranks

Tiny, robust rank-order filters that compete with neural networks

Juan Carlos del Río*

October 2026 (preprint)

Abstract

A median filter removes impulsive noise by sorting a few neighbouring samples and picking the middle one. Filters of this family (*rank-order* or *order-statistic* filters) need no multiplications, cannot be thrown off by a few wild samples, and their output never moves more than their input does. They were popular in the 1980s and 1990s, but they are hard to *train*: picking “the third largest value” does not change smoothly when the parameters change, so gradient descent gets no signal. We revisit a 1993 thesis on adaptive cascades of such filters with a simple idea: during training each sample is spread as a small *ramp* between its sorted neighbours, which makes the output smooth; for deployment the ramp is removed and the model is an exact rank filter again. We compare the resulting models (WOS-R) with convolutional networks, multilayer perceptrons and Transformers on signals, images, radar detection and federated learning. With 16 to 213 parameters and almost no multiplications, WOS-R comes within 0.5–1.2 dB of networks that are 160 to 1 400 times larger, and it is clearly better when the noise changes in ways not seen during training. It is not a universal replacement: for Gaussian noise or operators that need subtraction, networks remain better.

Abbreviations used throughout

WOS: weighted order statistic	CNN: convolutional neural network	dB: decibel (log ratio)
WOS-R: WOS trained with ramps	MLP: multilayer perceptron	SNR: signal-to-noise ratio
NEst: the 1993 structure (Sec. 3)	DnCNN: a standard denoising CNN	OOD: out of distribution

1 Why look again at rank filters?

Neural networks dominate signal and image restoration today. They are accurate, but large (thousands to millions of parameters), multiplication-heavy, and they can fail badly when the data drift away from what they saw in training. Rank-order filters sit at the other end of the spectrum: tiny, multiplication-free and robust by construction, but hand-designed.

In 1993 the author’s telecommunications engineering thesis [1], supervised by P. Salembier, studied how to *adapt* cascades of two such filters from data. Its main lesson was that parameters which only decide *which* sample is selected (a mask, a rank) cannot be adapted in a cascade, while parameters that change the *values* being sorted can. Thirty years later, with modern automatic differentiation and optimisers, we asked two questions: can these structures now be trained well, and are they useful next to today’s neural networks?

*Independent researcher, Madrid, Spain. jcdelrio100@gmail.com. Preprint DOI: [10.5281/zenodo.PPDOI](https://doi.org/10.5281/zenodo.PPDOI).

Main messages

1. A small trick, the *ramp*, makes rank filters trainable by gradient descent while the deployed model remains an exact rank filter with all its guarantees (Sec. 4).
2. Two layouts are useful: a *cascade* (stages in series, zero multiplications) and a *bank* (several cascades in parallel, combined by a weighted sum) (Sec. 4.3).
3. On impulsive noise, radar detection and fault-tolerant learning, these models match or beat networks that are hundreds of times larger, especially under conditions not seen in training (Sec. 5).

2 Rank-order filters in one page

Slide a short window over the signal (or a small square over the image), sort the samples inside it and output the sample found at a chosen position (rank). Figure 1 shows the median: an impulse ends up at one end of the sorted list and is never selected. Choosing rank 1 gives the minimum (*erosion*), the last rank gives the maximum (*dilation*); chaining an erosion and a dilation gives an *opening*, a classical shape filter [2, 3].

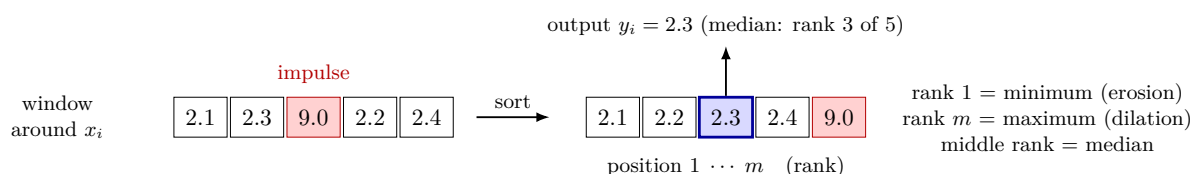


Figure 1: A rank filter. The window is sorted and one rank is selected. The impulse (9.0) is ignored.

Three properties make these filters attractive in practice:

- **No multiplications:** only comparisons, which are cheap in hardware.
- **Robustness:** if any 2 of the 5 samples of a median filter are corrupted, however wildly, the output still stays within the range of the clean samples. This tolerance (the *breakdown point*) is known in advance.
- **Stability:** if the input changes by at most ε everywhere, the output changes by at most ε (the filter is *1-Lipschitz*). This holds for any chain of rank filters, however long.

Their weakness is design. The output jumps from one sample to another when a parameter crosses a threshold, so the derivative with respect to the parameters is zero almost everywhere and gradient descent cannot learn them.

3 The 1993 structure: NEst

The thesis proposed a trainable stage, called NEst, that avoids the zero-gradient problem (Fig. 2). Each sample receives an offset b_j before sorting, and the sorted values are combined with weights h_k :

$$y_i = \sum_k |h_k| (z + b)_{(k)},$$

where $(z + b)_{(k)}$ is the k -th smallest value of the window after adding the offsets. Because the offsets change the *values*, the output varies smoothly with them, and the thesis adapted cascades of two NEst stages with the least-mean-squares (LMS) algorithm.

Re-implemented today with exact gradients and the Adam optimiser, a NEst cascade reaches the noise floor on the thesis' own example in 7s instead of 20s. However, when asked to recover

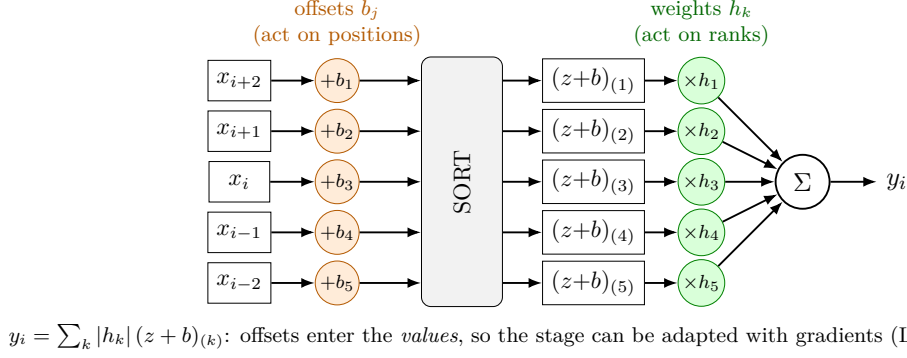


Figure 2: NEst stage (1993). Orange: offsets applied to window positions before sorting. Green: weights applied to the sorted values. With one weight equal to 1 and zero offsets it is a plain rank filter; with several weights it averages neighbouring ranks.

16 randomly chosen two-stage rank cascades, it succeeds in only 2–3 of them: its parameters can imitate a rank filter only approximately.

4 The proposal: WOS-R

4.1 Repetitions instead of masks

We use the *weighted order statistic* (WOS) filter [4, 5]: each sample of the window is copied w_j times before sorting, and the output is the sample found at a fraction r of the sorted list (Fig. 3a,b). A large w_j makes a position more influential; $w_j = 0$ removes it; equal weights and $r = \frac{1}{2}$ give the median; r near 0 or 1 gives erosion or dilation. The weights may be real numbers: any real weights define the same filter as some set of integers. One stage with a 7-sample window has only 8 parameters.

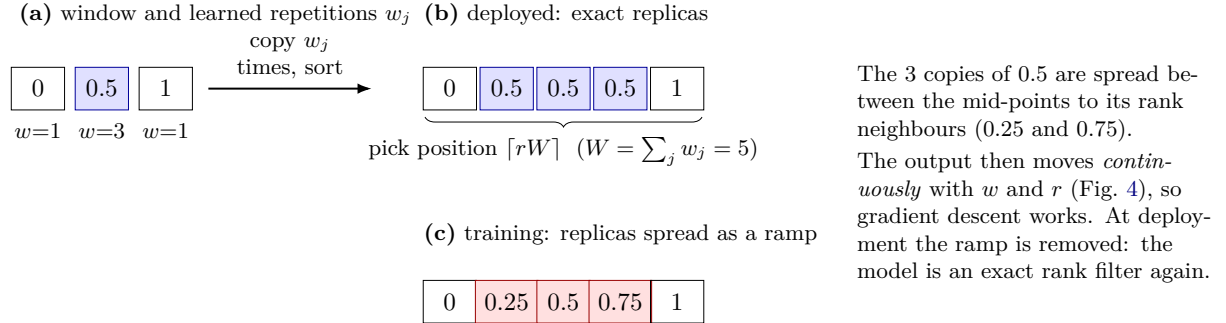


Figure 3: One WOS-R stage. (a) Each sample carries a learned repetition weight. (b) At deployment the copies are exact and one position of the sorted list is selected. (c) During training the copies are spread as a ramp between the neighbouring sorted values.

4.2 The ramp: smooth for training, exact for deployment

With exact copies, the output jumps between sample values when r or w change (blue curve in Fig. 4): zero gradient almost everywhere. During training we therefore replace the copies of each sample by a ramp that spans half the distance to its sorted neighbours (Fig. 3c). The output becomes a continuous, piecewise-linear function of the parameters (red curve), and ordinary gradient descent works. After training the ramp is switched off. The deployed model is again an exact rank filter, so the three properties of Section 2 hold for the model that is actually used.

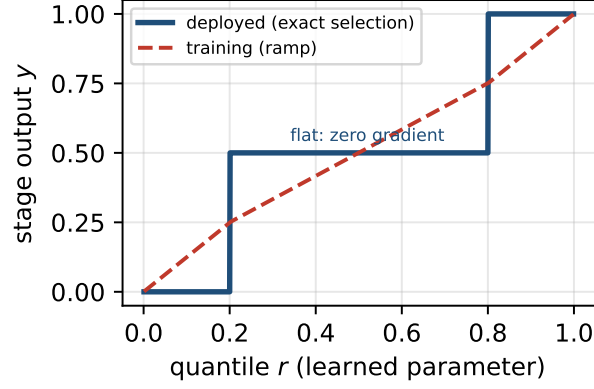


Figure 4: Output of the stage of Fig. 3 as the quantile r changes: staircase at deployment, continuous ramp during training.

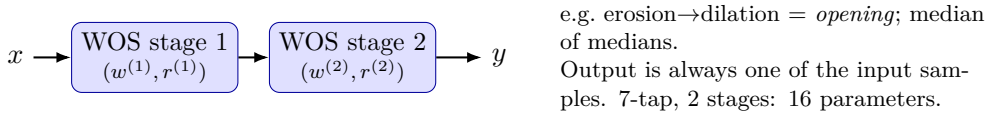
4.3 Cascade or bank?

Stages can be combined in two ways (Fig. 5, Table 1).

Cascade. Stages are applied one after the other, as in the 1993 thesis. The output is always one of the input samples; the model uses no multiplications and keeps every guarantee. It can express openings, closings, medians of medians and any chain of weighted medians.

Bank. C small cascades run in parallel on the same input, and their outputs are combined by a weighted sum $y = \sum_c a_c y_c + b$. This costs C multiplications per sample and buys expressiveness: the sum can subtract one operator from another (for instance identity minus opening, the *top-hat*) or average several rank filters. Robustness is kept, but the stability bound becomes $\sum_c |a_c|$ instead of 1. One practical detail matters: a bank must be trained with the ramp *gradually* reduced to zero during training. Otherwise the weighted sum learns to exploit the ramp itself, and the deployed bank can be very poor (20 dB worse on a simple median).

(a) **Cascade** (stages in series): *no multiplications*



(b) **Bank** (chains in parallel + weighted sum): C *multiplications*

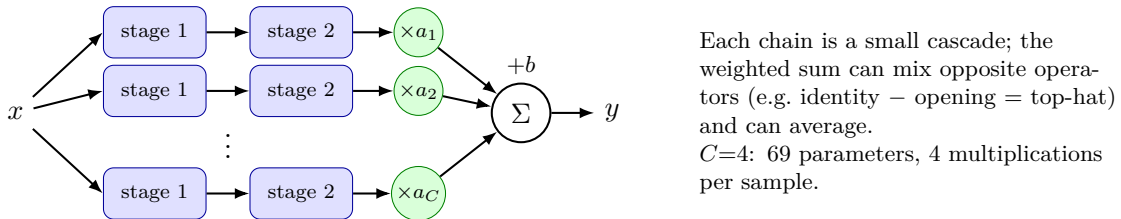


Figure 5: The two layouts used in this work.

Table 1: Cascade versus bank (1D, 7-sample windows, two stages per chain).

	Cascade	Bank of C chains
Structure	stages in series	C cascades in parallel + weighted sum
Parameters	16	69 ($C=4$), 137 ($C=8$)
Multiplications per sample	0	C
Output	always one input sample	weighted combination of C selected samples
Guarantees	robustness, stability bound 1, no multiplications	robustness; stability bound $\sum_c a_c $
Can represent	medians, erosion/dilation, openings, weighted medians	the same, plus signed combinations (top-hat) and coarse averaging
Training	ramp on, then switched off	ramp reduced gradually to zero
Error on 1D impulses	-18.7 ± 1.3 dB over 20 trainings (best -20.7)	-21.1 ± 0.5 dB over 6 trainings ($C=8$)
Sensitivity to initialisation	high: use a few restarts	low

5 How does it compare with neural networks?

We ran five tests on a 2-core CPU, comparing WOS-R with residual CNNs (small, medium and large), MLPs and small Transformers trained on the same data with the same loss. Table 2 gives the headline numbers; full tables are in the Appendix.

- **B1 – 1D signals with impulses.** Error in dB (lower is better; -20 dB means the remaining error has 1 % of the signal energy). The best bank (-21.1 dB, 137 parameters) is 1.2 dB behind the best CNN (22 081 parameters). A single 16-parameter cascade is less reliable: -18.7 ± 1.3 dB over 20 independent trainings; keeping the best of 4 short restarts (chosen on validation data) gives -19.6 dB, on a par with a 3 857-parameter CNN (-19.4 dB). When the impulses become twice as large as in training, the rank models degrade least and the Transformer breaks down.
- **B2 – What can each model represent?** We asked every model to imitate eleven known operators. When WOS-R finds a rank-type target it reproduces it *exactly*, which no network does; it cannot represent operators that need subtraction or squaring, which networks learn easily (Fig. 11).
- **B3 – Images with salt-and-pepper noise.** Quality in PSNR (peak signal-to-noise ratio, dB, higher is better) on 68 standard test photographs. A *switching* WOS-R, applied only to pixels that look like impulses, uses 52 parameters and is 0.5 dB behind a 74 593-parameter DnCNN at 50 % corrupted pixels. With a different kind of impulse never seen in training, the plain WOS-R is 5.6–6.7 dB *better* than the DnCNN (Fig. 9).
- **B4 – Radar detection.** A constant-false-alarm-rate (CFAR) detector decides whether a target is present from the surrounding range cells. A 34-parameter WOS-R is the best detector whenever other targets contaminate those cells; an MLP loses up to 4.3 dB and a Transformer stops detecting. Without interferers, the MLP is 0.3 dB better.
- **B5 – Federated learning with faulty nodes.** Twenty devices train a shared model and four send corrupted updates. A cascade of rank operations over time and over nodes keeps 75 % accuracy in the worst case (54 % when the devices hold very different data); a learned aggregator alone collapses to chance (10 %), but works well *after* the rank cascade.

Benchmark dashboards. Figures 6–8 show, for every model, the residual error in the training conditions and in unseen conditions, the number of parameters, the multiplications per output, the

Table 2: Headline results: best WOS-R against the best network in each test (from the Appendix tables).

Test	WOS-R (params)	score	Network (params)	score	Verdict
B1 impulses, 1D (dB, ↓)	bank (137)	−21.1	CNN (22 081)	−22.3	161× smaller, 1.2 dB worse
B1 impulses ×2 (OOD)	bank (137)	−17.8	CNN (22 081)	−17.4	WOS-R better; Transformer fails (+0.3)
B1 Gaussian noise	bank (137)	−18.2	CNN (22 081)	−20.4	network better
B3 salt & pepper 50 % (PSNR, ↑)	switching (52)	26.22	DnCNN (74 593)	26.74	1 434× smaller, 0.5 dB worse
B3 random impulses (OOD)	cascade (52)	24.46	DnCNN (74 593)	17.74	WOS-R +6.7 dB
B4 radar, 8 interferers (SNR needed, ↓)	WOS-R (34)	17.09	MLP (6 337)	20.35	WOS-R +3.3 dB
B5 federated, worst case (accuracy)	rank cascade (0)	0.753	learned (5 569)	0.100	learned only works after a rank cascade

CPU latency, the training curves and the time needed to converge. Colour marks the family: blue for rank-based models, orange for neural networks, green for classical filters. Three observations stand out. First, the rank models sit at the left of every cost axis (16–213 parameters, 0–8 multiplications) while staying within about 1 dB of the best network in the training conditions. Second, their error moves least when the conditions change (short segments in Figs. 6a and 7a). Third, they converge in 15–90s: WOS-R starts as a median filter, which is already a reasonable denoiser, so its training curve starts low (Fig. 6e). Their latency in PyTorch is not lower than that of small CNNs, because sorting is not what CPUs and deep-learning libraries are optimised for.

Cost. The savings are in operations: 0 to 8 multiplications per sample instead of 6 000–74 000, replaced by a few hundred comparisons. With standard PyTorch code, which is optimised for convolutions, WOS-R is not faster on a CPU (1.1–4.5 times slower); the gain would come from comparator hardware such as FPGAs or low-power DSPs.

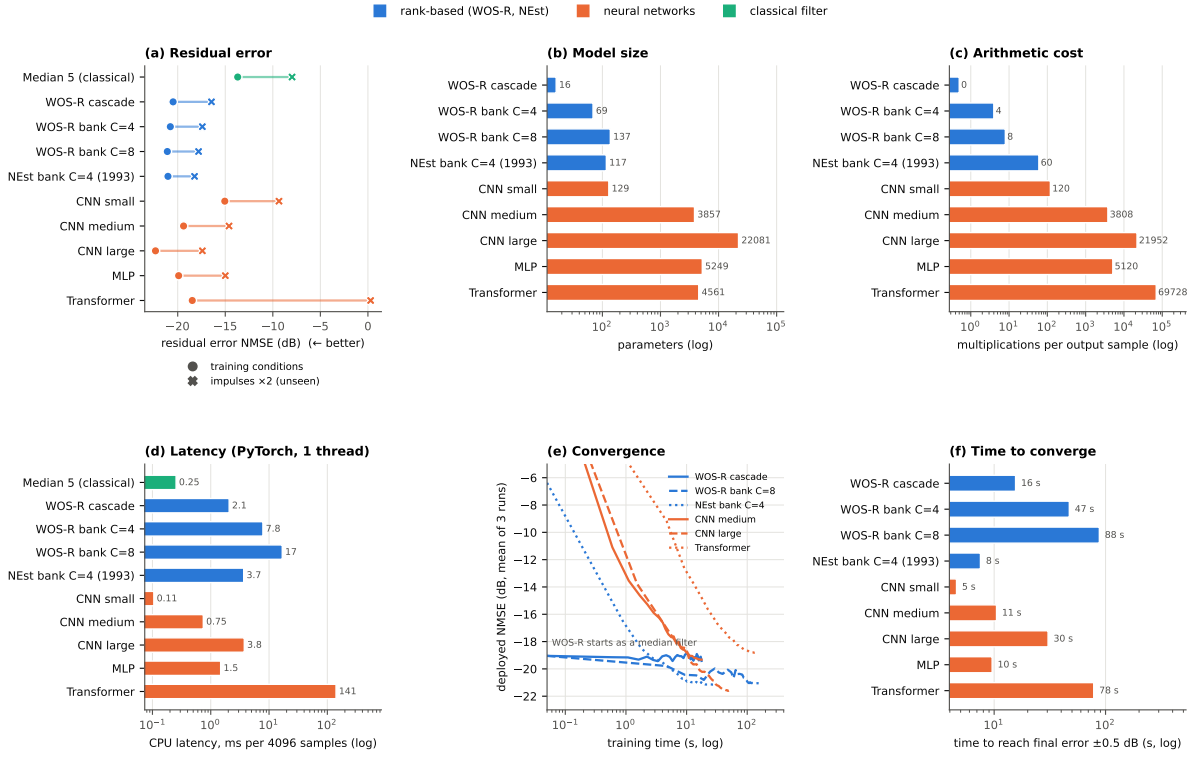


Figure 6: 1D impulsive denoising (B1). (a) Residual error (normalised mean squared error, NMSE; lower is better) in training conditions (o) and with impulses twice as large (x). (b) Parameters. (c) Multiplications per output sample. (d) CPU latency. (e) Error of the deployed model during training against training time (3 runs). (f) Time until the error stays within 0.5 dB of its final value.

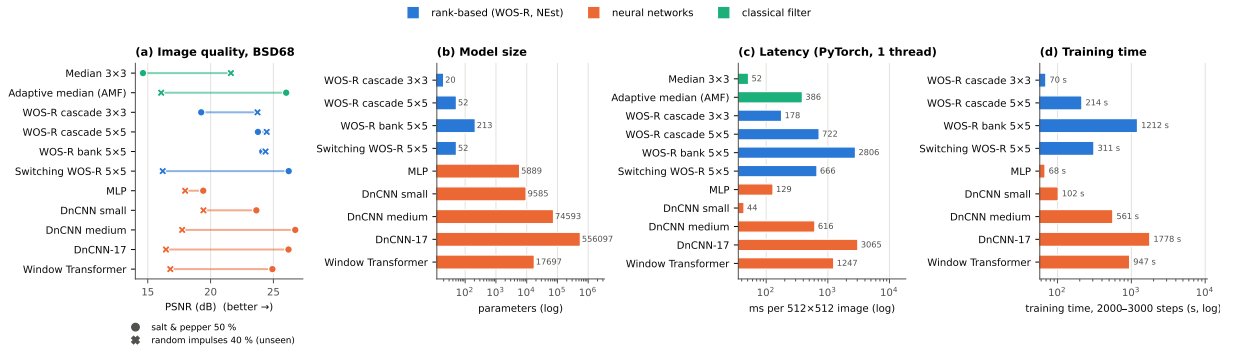


Figure 7: Image impulse denoising on 68 test photographs (B3). (a) Image quality (PSNR, higher is better) with 50% salt-and-pepper pixels (o, seen in training) and 40% random-valued impulses (x, never seen). (b) Parameters. (c) CPU latency per 512 $\times$ 512 image. (d) Training time.

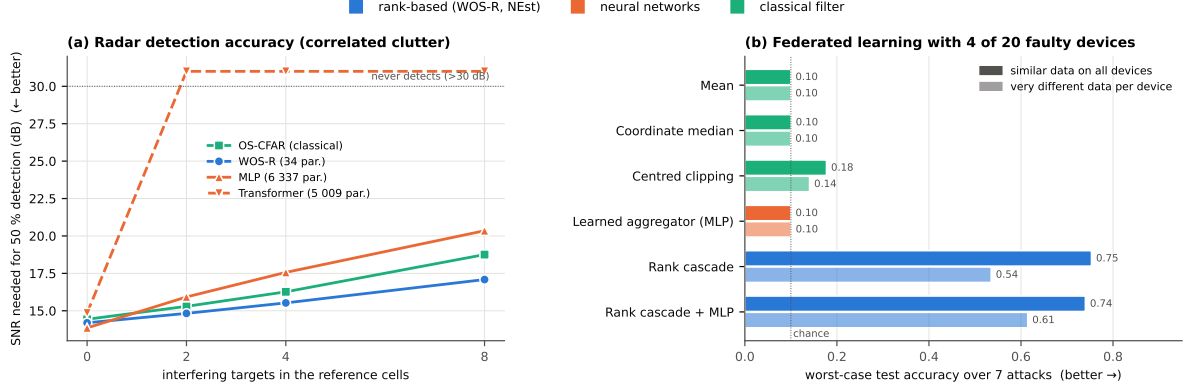


Figure 8: Accuracy in the two decision tasks. (a) Radar detection (B4): signal-to-noise ratio needed to detect half of the targets as interfering targets are added (lower is better; the Transformer never reaches 50 % with interferers). (b) Federated learning (B5): worst test accuracy over seven types of faulty devices.

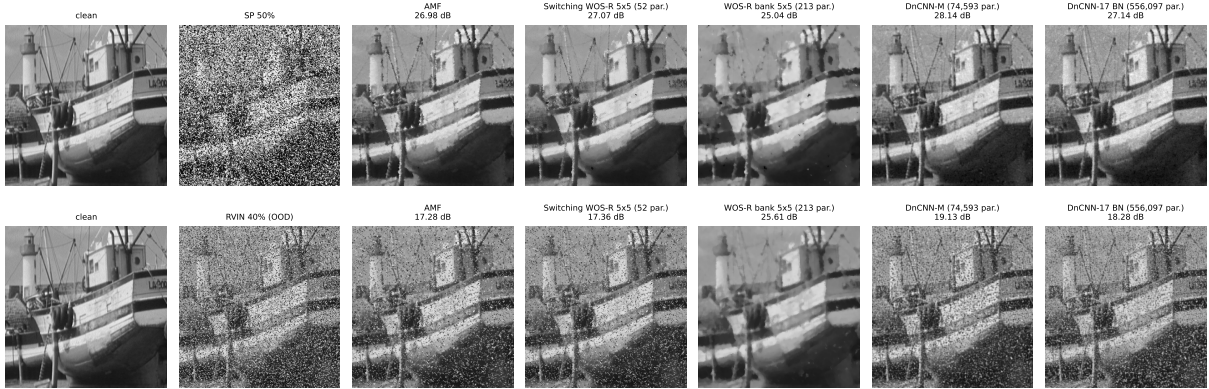


Figure 9: Image test (“boat”, crop). Top: 50 % salt-and-pepper pixels (seen in training). Bottom: 40 % random-valued impulses (never seen). AMF is the classical adaptive median filter. Numbers are PSNR of the full image.

6 Are stages and window size hyper-parameters?

In the experiments above they were fixed by hand (for instance 2 stages of 7 samples). They do not have to be: the *maximum* depth, window and bank size are hyper-parameters, but the structure actually used can be learned during training without leaving the rank-filter family, because every unnecessary piece has a natural “off” state:

- a **stage** becomes the identity when the weight of its centre sample exceeds the larger of r and $1 - r$ (the centre is then always selected), so it can simply be removed;
- a **tap** disappears when its repetition weight goes to zero, which shortens the window;
- a **chain** of a bank disappears when its mixing weight a_c goes to zero.

We trained an over-sized cascade (4 stages of 9 samples, extra stages starting as the identity) with two small penalties that push stages towards the identity and weights towards zero, then removed every stage and tap whose removal did not change the error on separate validation data (Table 3).

Table 3: Structure learned from an over-sized cascade (4 stages $\times$ 9 samples). Error above the best achievable, in dB; 2 training runs per target (3 for B1).

Target	True structure	Learned structure	Result
Median of 7	1 stage, 7 samples	1 stage, 7 samples (all runs)	exact (0.0 dB)
Opening (erosion $\rightarrow$ dilation)	2 stages of 5	3–4 smaller stages, equivalent	exact in 3 of 4 runs; the fixed 2×7 cascade never found it
Rank cascade #1	median of 5, then erosion of 3	2 stages, the second an erosion of 3	exact in 2 of 4 runs
1D impulses (B1)	unknown	1–2 stages, 10–16 parameters	–19.1 dB (fixed 2×7 : –18.7)
Bank of 8 chains (B1)	unknown	6–7 chains kept	no loss (–20.8 dB)

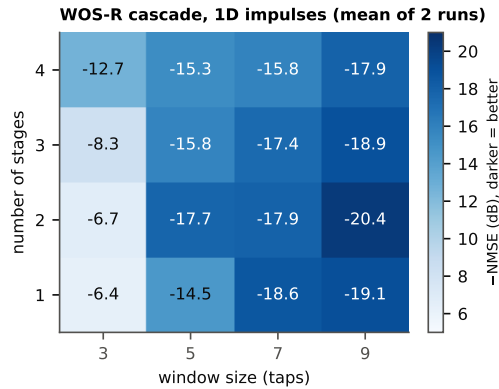


Figure 10: Hand-tuning the same two hyper-parameters: error of a WOS-R cascade on 1D impulses for every combination of depth and window (darker is better).

Two lessons follow. The structure can be learned: the median and the second stage of cascade #1 are recovered exactly, and the B1 model shrinks to 1–2 stages without losing accuracy. More surprisingly, *starting larger helps the optimiser*: the opening, which the fixed 2-stage cascade never found, is found by the over-sized one as an equivalent chain of smaller filters. For comparison, a plain grid search over depth and window (Fig. 10) costs 16 trainings (one per configuration) and

still depends on luck: deeper cascades started at random get worse, and a single 2×7 cascade varies by up to 4dB between runs. Since one training takes about 20s, the practical recipe is: start over-sized with extra stages at the identity, prune, and keep the best of a few restarts.

7 When should you use it?

WOS-R is a good choice	A neural network is a better choice
impulsive or heavy-tailed noise, especially of unknown size or type	Gaussian noise at the level seen in training
radar detection with interfering targets	more corrupted samples than the filter can tolerate
robust aggregation of data from unreliable sources (as a front-end)	operators that need subtraction or squaring
very small budgets: < 250 parameters, almost no multiplications	highest possible accuracy with GPU training

A practical recipe that emerges from the tests is *rank first, learn second*: a WOS-R cascade bounds the influence of corrupted inputs, and a small learned model placed after it adds accuracy.

Limitations. All training ran on a CPU, images used a single training run per model, and the networks are smaller and less trained than published GPU models, so their accuracy in the training conditions is underestimated. The radar data are simulated. Training does not always find the best rank filter: a fixed two-stage cascade recovered 9 of 16 random cascades (against 2–3 for NEst), and a single cascade is sensitive to its random start (Sec. 6); banks and over-sized cascades are much more reliable.

8 Conclusion

A forty-year-old family of filters becomes trainable with a simple device: let each sample be a ramp while learning, and a single sample when deployed. The resulting models are tiny, need almost no multiplications, keep provable robustness, and hold their own against neural networks hundreds of times larger, above all when the data change. They are not general approximators; combining them with small linear or learned stages is the natural next step.

Code and data. All code, result files and a detailed technical version of this work are available at https://github.com/GITHUB_USER/wosr-rank-filters and archived at [doi:10.5281/zenodo.SWDOI](https://doi.org/10.5281/zenodo.SWDOI).

Acknowledgement. To Philippe Salembier, who supervised the 1993 thesis that started this work.

References

- [1] Juan Carlos del Río Romero. Adaptación de estructuras y filtros de orden (adaptation of structures and order filters). Master’s thesis, Escola Tècnica Superior d’Enginyers de Telecomunicació de Barcelona, Universitat Politècnica de Catalunya, 1993. Supervisor: P. Salembier.
- [2] Ioannis Pitas and Anastasios N. Venetsanopoulos. *Nonlinear Digital Filters: Principles and Applications*. Kluwer Academic Publishers, 1990.

- [3] Petros Maragos. A representation theory for morphological image and signal processing. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(6):586–599, 1989.
- [4] D. R. K. Brownrigg. The weighted median filter. *Communications of the ACM*, 27(8):807–818, 1984.
- [5] Lin Yin, Ruikang Yang, Moncef Gabbouj, and Yrjö Neuvo. Weighted median filters: a tutorial. *IEEE Transactions on Circuits and Systems II*, 43(3):157–192, 1996.
- [6] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafaël Pinot, and John Stephan. Fixing by mixing: a recipe for optimal Byzantine ML under heterogeneity. In *Proc. AISTATS*, 2023.

A Detailed results

Abbreviations in the tables: SP salt-and-pepper; RVIN random-valued impulse noise; OOD out of distribution; NMSE normalised mean squared error; PSNR peak signal-to-noise ratio; SSIM structural similarity (0–1, higher is better); AMF adaptive median filter; CNN-S/M/L small-/medium-/large CNN; CFAR constant false-alarm rate; P_d , P_{fa} probability of detection / false alarm; CW coordinate-wise; NNM nearest-neighbour mixing; AMO adaptive minimax ordered weighted average; Med_t3 temporal median of the last 3 updates; ALIE “a little is enough” attack; IPM inner-product manipulation attack; SDC silent data corruption. Mults and Comps: multiplications and comparisons per output sample. “Ramp only” rows are banks trained without the gradual reduction of the ramp (Sec. 4.3).

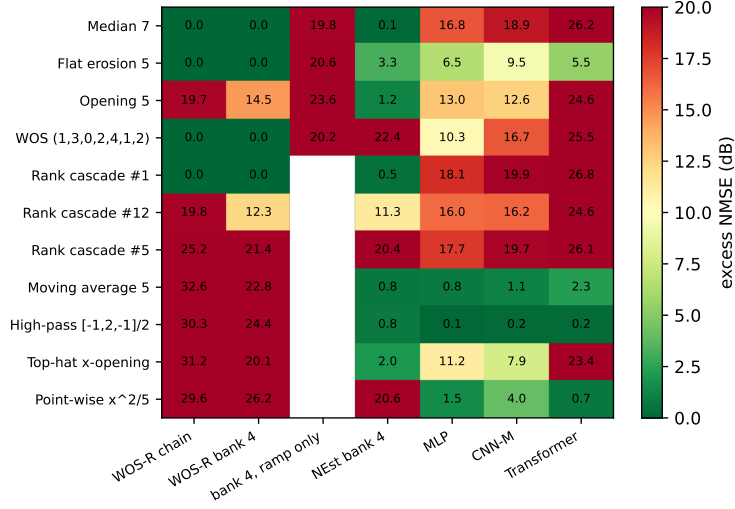


Figure 11: B2: how far each model stays from each target operator (dB above the best achievable; green represented, red not represented).

Table 4: B1: 1D restoration, normalised mean squared error (NMSE) in dB (mean $\pm$ std over 3 seeds; lower is better). Impulsive models are trained with 10 % impulses; OOD (out-of-distribution) columns change impulse rate or amplitude. NEst: 1993 thesis structure; CNN-S/M/L: small/medium/large residual CNN. Gaussian models are trained with $\sigma = 0.2$. Params, multiply-accumulates and comparisons per output sample; ms: CPU latency for 4096 samples (1 thread). Best per column in bold.

Model	Params	Mults	Comps	ms	Impulsive			Gaussian	
					in-dist	OOD $p=25\%$	OOD $\times 2$	$\sigma=0.2$	$\sigma=0.4$ (OOD)
Median 5	0	0	15	0.3	-13.7 ± 1.2	-3.6 ± 0.3	-8.0 ± 1.3	-14.9 ± 0.3	-8.9 ± 0.4
WOS-R chain 2×7	16	0	42	2.1	-20.5 ± 0.1	-11.3 ± 0.3	-16.5 ± 1.7	-16.4 ± 0.3	-11.1 ± 0.4
WOS-R bank $C=4$	69	4	168	7.8	-20.8 ± 0.5	-12.1 ± 0.5	-17.4 ± 1.1	-17.9 ± 0.2	-12.4 ± 0.4
WOS-R bank $C=8$	137	8	336	16.9	-21.1 ± 0.6	-12.4 ± 0.6	-17.8 ± 0.9	-18.2 ± 0.4	-12.7 ± 0.4
ablation: bank $C=4$, ramp only	69	4	168	7.8	-19.9 ± 0.9	-11.1 ± 1.6	-16.1 ± 1.4	—	—
ablation: bank $C=8$, ramp only	137	8	336	16.9	-19.8 ± 0.9	-10.9 ± 1.7	-16.5 ± 1.2	—	—
NEst bank $C=4$	117	60	168	3.7	-21.0 ± 0.4	-12.3 ± 0.3	-18.2 ± 0.9	-18.5 ± 0.2	-12.5 ± 0.3
CNN-S	129	120	0	0.1	-15.0 ± 0.7	-8.3 ± 0.2	-9.4 ± 0.9	-19.1 ± 0.3	-11.3 ± 0.3
CNN-M	3857	3808	0	0.7	-19.4 ± 0.4	-12.9 ± 1.0	-14.6 ± 0.6	-20.4 ± 0.3	-12.1 ± 0.6
CNN-L	22081	21952	0	3.8	-22.3 ± 0.6	-15.5 ± 0.7	-17.4 ± 0.0	-20.4 ± 0.2	-11.6 ± 0.7
MLP (window 15)	5249	5120	0	1.5	-19.9 ± 0.3	-12.9 ± 0.5	-15.0 ± 0.1	-19.7 ± 0.3	-12.5 ± 0.4
Transformer	4561	69728	0	141.0	-18.5 ± 0.3	-11.1 ± 0.2	0.3 ± 0.3	-16.5 ± 0.3	-9.6 ± 0.4

Table 5: B2 coverage: excess normalised mean squared error (NMSE, dB) above the noise floor of the true operator (input $U[-5, 5]$, SNR 35 dB, mean of 2 seeds). Bold: ≤ 1 dB (operator represented); red: > 6 dB (not represented). Parameters in the last row.

Target	Class	WOS-R chain	WOS-R bank 4	bank 4, ramp only	NEst bank 4	MLP	CNN-M	Transformer
Median 7	rank	0.0	0.0	19.8	0.1	16.8	18.9	26.2
Flat erosion 5	morphological	0.0	0.0	20.6	3.3	6.5	9.5	5.5
Opening 5	morph., 2 stages	19.7	14.5	23.6	1.2	13.0	12.6	24.6
WOS (1, 3, 0, 2, 4, 1, 2)	WOS	0.0	0.0	20.2	22.4	10.3	16.7	25.5
Rank cascade #1	rank cascade	0.0	0.0	—	0.5	18.1	19.9	26.8
Rank cascade #12	rank cascade	19.8	12.3	—	11.3	16.0	16.2	24.6
Rank cascade #5	rank cascade	25.2	21.4	—	20.4	17.7	19.7	26.1
Moving average 5	linear, $h \geq 0$	32.6	22.8	—	0.8	0.8	1.1	2.3
High-pass $[-1, 2, -1]/2$	linear, signed	30.3	24.4	—	0.8	0.1	0.2	0.2
Top-hat x -opening	non-increasing	31.2	20.1	—	2.0	11.2	7.9	23.4
Point-wise $x^2/5$	not translation-inv.	29.6	26.2	—	20.6	1.5	4.0	0.7
Params		16	69	69	117	5249	3857	4561

Table 6: B3: blind impulse denoising on BSD68, peak signal-to-noise ratio (PSNR, dB; higher is better) / structural similarity (SSIM). Training: salt-and-pepper (SP) with density $\sim U[0.1, 0.6]$. Out of distribution (OOD): SP 70 %, random-valued impulse noise (RVIN). AMF: adaptive median filter; switching: WOS-R applied only to pixels that are an extreme of their 3×3 window. Mults and comparisons per pixel; latency in ms for a 512×512 image (1 CPU thread). Best per column in bold.

Model	Params	Mults	Comps	ms	SP 10%	SP 30%	SP 50%	SP 70%	RVIN 20%	RVIN 40%
Median 3×3	0	0	36	52	28.26/0.851	21.94/0.682	14.62/0.268	9.58/0.069	26.70/0.802	21.62/0.552
Median 5×5	0	0	125	149	25.79/0.731	24.90/0.711	21.49/0.617	13.48/0.226	25.36/0.718	23.59/0.637
Adaptive median (AMF, 7)	0	0	var.	386	32.29/0.944	29.07/0.909	26.02/0.839	20.06/0.647	21.05/0.524	16.07/0.260
WOS-R 3×3 , 2 stages	20	0	72	178	27.37/0.812	25.50/0.774	19.25/0.562	11.90/0.154	26.63/0.793	23.73/0.668
WOS-R 5×5 , 2 stages	52	0	250	722	26.65/0.771	25.83/0.751	23.77/0.705	15.83/0.422	26.19/0.757	24.46/0.696
WOS-R bank $C=4$ (3×3)	85	4	288	489	26.46/0.785	25.41/0.755	20.47/0.535	13.37/0.162	25.78/0.770	23.38/0.661
WOS-R bank $C=4$ (5×5)	213	4	1000	2806	26.60/0.770	25.89/0.752	24.11/0.699	16.85/0.372	26.07/0.756	24.37/0.696
ablation: bank $C=4$ (3×3), ramp only	85	4	288	489	26.98/0.801	25.62/0.766	20.22/0.532	13.11/0.157	26.22/0.784	23.55/0.667
ablation: bank $C=4$ (5×5), ramp only	213	4	1000	2806	26.74/0.776	25.98/0.757	24.08/0.704	16.55/0.369	26.20/0.762	24.43/0.700
Switching WOS-R 3×3	20	0	90	240	31.18/0.933	27.88/0.894	20.19/0.657	12.53/0.197	20.98/0.513	16.17/0.257
Switching WOS-R 5×5	52	0	268	666	30.52/0.923	28.88/0.902	26.22/0.840	18.48/0.604	20.89/0.505	16.19/0.253
Switching WOS-R bank $C=4$ (3×3)	85	4	306	726	29.78/0.885	27.76/0.843	21.87/0.602	14.82/0.218	20.82/0.501	16.11/0.253
MLP 5×5	5889	5760	0	129	30.09/0.892	24.37/0.642	19.42/0.384	15.44/0.200	21.87/0.513	17.98/0.317
DnCNN-S (6 layers, 16 ch)	9585	9504	0	44	29.27/0.868	26.36/0.734	23.64/0.578	19.94/0.375	23.66/0.628	19.43/0.424
DnCNN-M (10 layers, 32 ch)	74593	74304	0	616	33.65/0.950	29.97/0.881	26.74/0.761	21.92/0.521	22.08/0.530	17.74/0.336
DnCNN-17 (BN, 64 ch)	556097	554112	0	3065	31.41/0.911	29.21/0.835	26.20/0.713	21.74/0.502	20.96/0.463	16.45/0.280
Window Transformer	17697	25152	0	1247	30.37/0.915	28.01/0.834	24.92/0.681	20.17/0.437	20.28/0.447	16.80/0.291

Table 7: B4: constant-false-alarm-rate (CFAR) detection in K-distributed clutter with range-correlated texture (AR(1) coefficient ρ), false-alarm probability $P_{fa} = 10^{-4}$ calibrated per detector. Signal-to-noise ratio (SNR, dB) for detection probability $P_d = 0.5$ with 0/2/4/8 interfering targets in the reference window (lower is better); learned detectors: mean of 2 seeds, trained with the same Neyman–Pearson objective.

Detector	Params	$\rho = 0$				$\rho = 0.9$			
		0	2	4	8	0	2	4	8
OS-CFAR ($k=24$)	0	15.46	16.41	17.33	19.91	14.43	15.30	16.27	18.75
GO-OS ($k=11$)	0	15.76	16.63	17.50	20.09	14.91	15.72	16.50	18.85
WOS-R (32 repetitions)	34	15.55	16.29	16.95	18.38	14.20	14.83	15.53	17.09
MLP (log-equivariant)	6 337	15.22	17.64	19.37	22.69	13.86	15.92	17.57	20.35
Transformer (log-equivariant)	5 009	15.55	> 30	> 30	> 30	14.85	> 30	> 30	> 30

Table 8: B5: Byzantine/fault-robust aggregation (Fashion-MNIST, $n=20$ nodes, $f=4$ faulty, 300 steps, mean of 2 seeds). Test accuracy under each attack with heterogeneous data (Dirichlet $\alpha=0.1$), and worst case over attacks for $\alpha \in \{1, 0.1\}$. CW: coordinate-wise; Med_t3: per-node temporal median of the last 3 submissions; NNM: nearest-neighbour mixing [6]; AMO: adaptive minimax ordered weighted average (OWA). Attacks: ALIE (“a little is enough”), IPM (inner-product manipulation), burst-8 (faults persisting 8 steps), SDC (silent data corruption). [†]AMO uses a precomputed library of 5 OWA weight vectors (minimax second-order cone program), no training.

Aggregator	Params	$\alpha = 0.1$							worst case	
		none	sign-flip	ALIE	IPM	burst-8	SDC	Gauss+burst	$\alpha=1$	$\alpha=0.1$
Mean	0	0.814	0.414	0.787	0.789	0.100	0.100	0.100	0.100	0.100
CW median	0	0.603	0.599	0.565	0.465	0.244	0.683	0.100	0.100	0.100
CW trimmed mean	0	0.728	0.642	0.728	0.372	0.100	0.707	0.100	0.100	0.100
NNM + CW trim	0	0.793	0.727	0.714	0.695	0.100	0.803	0.100	0.100	0.100
Centred clipping	0	0.806	0.681	0.771	0.693	0.707	0.803	0.141	0.177	0.141
AMO (minimax OWA)	0 [†]	0.591	0.551	0.610	0.410	0.247	0.602	0.100	0.100	0.100
Med _t 3 → AMO	0 [†]	0.576	0.502	0.527	0.264	0.542	0.548	0.539	0.691	0.264
Med _t 3 → NNM → CW trim	0	0.779	0.672	0.772	0.535	0.782	0.773	0.762	0.753	0.535
Sorted-MLP aggregator	5 569	0.673	0.646	0.718	0.380	0.101	0.113	0.100	0.100	0.100
Med _t 3 → NNM → sorted-MLP	5 569	0.777	0.703	0.767	0.615	0.780	0.775	0.773	0.740	0.615